Informe Entrega 2 - Paralelo 2 - Grupo 4

Danniel Guardia, Javier Loyola, Matias Ibañez, Joaquín Hormaechea
dguardia@usm.cl, jloyola@usm.cl, mibanez@usm.cl, jhormaechea@usm.cl
UTFSM Casa Central, Valparaíso, Chile

I. Introducción

En Chile, más del 60 % de las viviendas carece de un aislamiento térmico adecuado. Esta realidad no solo impacta al medioambiente, sino que golpea directamente el bolsillo de las familias por los altos costos de calefacción. Hoy, frente a las nuevas exigencias de la Calificación Energética de Viviendas (CEV), existe una necesidad clara: las personas necesitan herramientas accesibles que les permitan tomar decisiones verdaderamente informadas para mejorar el confort de sus hogares, abarcando elementos clave como muros, techos, pisos, ventanas y puertas.

Este proyecto se inserta en el desafío de la eficiencia energética residencial, tomando como inspiración el sistema "Bio Planner", una iniciativa real codesarrollada con la oficina de arquitectura Acua Plan y presentada en la Feria de Software (FESW) 2025. Nuestro objetivo es claro: construir un modelo matemático que empodere al usuario. Buscamos determinar la combinación óptima de soluciones constructivas que minimice la inversión económica total en materiales y, al mismo tiempo, garantice un estándar térmico global de excelencia, respetando la transmitancia máxima permitida ($U^{\max}$), siempre adaptándonos a las restricciones particulares de cada familia.

Para trasladar esta problemática real a un modelo matemático estructurado y viable, hemos definido cinco supuestos fundamentales:

1. **Dimensiones estáticas del hogar:** Partimos de la base de que las áreas de cada elemento constructivo son constantes y plenamente conocidas. El modelo optimiza la elección de materiales, pero en ningún caso altera la escala ni la arquitectura original de la vivienda.
2. **Decisiones únicas y definitivas:** Para facilitar la instalación práctica y asegurar la viabilidad del proyecto, cada elemento modificable utilizará exactamente una alternativa. Descartamos por completo la combinación de soluciones parciales en un mismo sector.
3. **Independencia térmica entre zonas:** Abordamos el desempeño térmico asumiendo que la transmitancia se aporta de forma individual por cada elemento. Es decir, el aislamiento de una zona no altera a las adyacentes, independiente de su distancia o superficie de contacto.
4. **Confiabilidad del catálogo:** Trabajamos con la certeza de que los costos y transmitancias entregados son exactos y representan fielmente el desempeño térmico real de los materiales. Asimismo, garantizamos que todas las alternativas ofrecidas en el catálogo son técnicamente factibles para el elemento al que están asignadas.
5. **Foco en la inversión directa:** Definimos el costo total de inversión basándonos de manera exclusiva en la compra de los materiales. Para aislar el problema y mantener la precisión del modelo, dejamos fuera costos externos operativos como la mano de obra, el transporte o el mantenimiento a largo plazo.

II. Modelamiento del problema

1. **Conjuntos e índices**

- I: Conjunto de todos los elementos constructivos de la vivienda.
- I^F: Subconjunto de elementos constructivos fijos ($I^F \subseteq I$).
- I^O: Subconjunto de elementos constructivos optimizables ($I^O \subseteq I$).
- J_i: Conjunto de alternativas constructivas (catálogo) disponibles para el elemento i.
- i: Índice que representa un elemento constructivo específico.
- j: Índice que representa una alternativa constructiva específica para un elemento.

2. **Parámetros**
   - $A_i > 0$: Área del elemento constructivo i [m^2].
   - $C_{ij} \geq 0$: Costo unitario de la alternativa j para el elemento i [\$/m^2].
   - $U_{ij} > 0$: Transmitancia térmica de la alternativa j para el elemento i [$W/(m^2K)$].
   - $C_i^F \geq 0$: Costo unitario del material predefinido para el elemento fijo i [\$/m^2].
   - $U_i^F > 0$: Transmitancia térmica del material predefinido para el elemento fijo i [$W/(m^2K)$].
   - $U^{\max} > 0$: Límite máximo permitido para la transmitancia térmica global promedio de la vivienda [$W/(m^2K)$].

3. **Variables de decisión**
   - $x_{ij} = 1$ si se selecciona la alternativa constructiva j para el elemento optimizable i, y 0 en caso contrario.

4. **Función objetivo**
   - $\min Z = \sum_{i \in I^O} \sum_{j \in J_i} A_i C_{ij} x_{ij} + \sum_{i \in I^F} A_i C_i^F$

     La función minimiza el costo total de inversión en materiales de la vivienda. El primer término calcula el costo total de los elementos optimizables, multiplicando el área de cada elemento libre por el precio unitario de la alternativa seleccionada. El segundo término suma el costo constante de todos los elementos fijos (área multiplicada por su costo unitario predefinido), garantizando la consistencia dimensional en la métrica monetaria total.

5. **Restricciones**
   - $\sum_{j \in J_i} x_{ij} = 1, \quad \forall i \in I^O$
     Significado operativo: Garantiza que a cada elemento constructivo optimizable se le asigne exactamente una, y solo una, solución constructiva del catálogo, evitando viviendas incompletas o la superposición de materiales en un mismo sector.
   - $\sum_{i \in I^F} A_i U_i^F + \sum_{i \in I^O} \sum_{j \in J_i} A_i U_{ij} x_{ij} \leq U^{\max} \sum_{i \in I} A_i$
     Significado físico: Asegura que el balance térmico de la vivienda cumpla con la normativa. Exige que la suma de la transmitancia aportada por los elementos fijos más la transmitancia de las alternativas seleccionadas (ambas ponderadas por el área de sus respectivos elementos) sea menor o igual al umbral máximo permitido distribuido en el área total de la estructura.

6. **Condiciones de dominio**
   - $x_{ij} \in \{0, 1\}, \quad \forall i \in I^O, j \in J_i$

III. Generador de Instancias

El procedimiento para la generación de las 15 instancias factibles solicitadas se implementó en Python, estructurando el código de acuerdo con las especificaciones del problema.

1. Algoritmo y parámetros configurables

- **Entradas:** Tipo de instancia (Pequeña, Mediana o Grande) y una semilla de aleatoriedad (`seed = 50`).
- **Salidas:** Un diccionario estructurado con las dimensiones físicas, el catálogo de alternativas, la configuración de los elementos (fijos o libres) y el umbral U^{max}.
- **Parámetros configurables:** Se definieron rangos físicos para la vivienda (ej. altura entre 2.3 y 2.8 m), rangos de transmitancia térmica ($0{,}2$ a $5{,}0\,W/m^2K$) y rangos de costo de materiales (5000 a $65000\,\$/m^2$).

2. Distribuciones y aleatoriedad reproducible
   - Se utilizó la librería estándar `random` fijando la semilla al inicio de la ejecución para garantizar que los resultados sean exactamente reproducibles.
   - Las dimensiones físicas, áreas, valores de transmitancia U y costos se generaron utilizando distribuciones uniformes continuas (`random.uniform`).
   - La cantidad de categorías activas, alternativas por catálogo y número de elementos libres se definieron mediante distribuciones uniformes discretas (`random.randint`), acotadas estrictamente por los rangos de la Tabla 1 del enunciado.
3. Tamaño, estructura y almacenamiento
   - El tamaño del problema escala con el tipo de instancia. Por ejemplo, las instancias grandes consideran 5 categorías, entre 26 y 50 alternativas de materiales, y entre 9 y 15 elementos libres a optimizar.
   - Los datos se almacenan en tiempo de ejecución mediante diccionarios y listas, lo que permite asociar fácilmente un identificador, área y estado ("fijo.º "libre") a cada elemento constructivo.
4. Condiciones para garantizar factibilidad técnica
   - Para asegurar que el modelo matemático no enfrente un espacio de soluciones vacío, se implementó una rutina que simula el escenario de máxima eficiencia térmica posible.
   - Para cada elemento libre, se busca temporalmente en el catálogo la alternativa con menor transmitancia (mejor aislamiento) y se calcula la transmitancia global equivalente mínima (U_{eq_min}).
5. Procedimiento de validación
   - El parámetro de exigencia U^{max} se calcula añadiendo un margen aleatorio (holgura entre $10\,\%$ y $35\,\%$) a la diferencia entre el peor escenario de aislamiento y el escenario U_{eq_min}.
   - Toda instancia se valida verificando estrictamente la condición lógica $U^{max} \geq U_{eq_min}$, garantizando que los materiales más eficientes siempre cumplan la norma.
6. Tabla resumen de instancias
   - A continuación, se detallan las propiedades principales de las 15 instancias de prueba generadas:

ID Instancia	Tipo	Categorías	Elem. Libres	U^{max}	Factible
Pequena_1	Pequena	3	4	3.821	Sí
Pequena_2	Pequena	3	4	2.711	Sí
Pequena_3	Pequena	3	4	2.241	Sí
Pequena_4	Pequena	4	3	2.502	Sí
Pequena_5	Pequena	3	2	1.762	Sí
Mediana_6	Mediana	5	7	1.567	Sí
Mediana_7	Mediana	5	5	2.422	Sí
Mediana_8	Mediana	5	5	2.463	Sí
Mediana_9	Mediana	5	7	2.501	Sí
Mediana_10	Mediana	5	6	1.788	Sí
Grande_11	Grande	5	11	1.891	Sí
Grande_12	Grande	5	14	2.135	Sí
Grande_13	Grande	5	13	2.478	Sí
Grande_14	Grande	5	13	1.527	Sí
Grande_15	Grande	5	13	2.16	Sí

Cuadro I
RESUMEN DE LAS 15 INSTANCIAS FACTIBLES GENERADAS.

7. Código completo del generador

- Se adjunta el script ejecutado para la creación y validación de los datos.

```
import random

tiposInstancias = ["Pequena", "Mediana", "Grande"]
categorias = ["Muro", "Techo", "Piso", "Ventana", "Puerta"]
totalInstancias = 15

#Cambiar la semilla para obtener diferentes instancias
semilla = 50
random.seed(semilla)

instanciasPorTipo = 5

uMax = {
    "holgura_min": 0.1,
    "holgura_max": 0.35
}

categoriasDeElementos = {
    "Pequena": (3, 4),
    "Mediana": (5, 5),
    "Grande": (5, 5)
}

alternativasPorCategoria = {
    "Pequena": (5, 10),
    "Mediana": (11, 25),
    "Grande": (26, 50)
}

elementosLibresPorTipo = {
    "Pequena": (2, 4),
    "Mediana": (5, 8),
    "Grande": (9, 15)
}

largoCasa = {"min": 5.0, "max": 12.0}
anchoCasa = {"min": 5.0, "max": 10.0}
alturaCasa = {"min": 2.3, "max": 2.8}
areaCasa = {"min": 25.0, "max": 120.0}

areaPorCategoria = {
    "Muro": (8.0, 25.0),
    "Techo": (25.0, 120.0),
    "Piso": (25.0, 120.0),
    "Ventana": (1.0, 3.5),
    "Puerta": (1.6, 2.5)
}

```

```
transmitancia = {"min": 0.2, "max": 5.0}
costo = {"min": 5000, "max": 65000}

def generarVivienda():
    largo = random.uniform(largoCasa["min"], largoCasa["max"])
    ancho = random.uniform(anchoCasa["min"], anchoCasa["max"])
    altura = random.uniform(alturaCasa["min"], alturaCasa["max"])
    area = largo * ancho
    volumen = area * altura
    return round(largo, 2), round(ancho, 2), round(altura, 2), round(area, 2), round(volumen, 2)

def generarElementos(tipoInstancia, largo, ancho, altura):
    elementos = []
    num_cat = random.randint(categoriasDeElementos[tipoInstancia][0], categoriasDeElementos[tipoInstancia
        ][1])
    cats_seleccionadas = random.sample(categorias, num_cat)

    min_libres = elementosLibresPorTipo[tipoInstancia][0]
    max_libres = elementosLibresPorTipo[tipoInstancia][1]
    total_libres_objetivo = random.randint(min_libres, max_libres)

    cantidad_total_elementos = max(15, total_libres_objetivo + 5)

    for i in range(cantidad_total_elementos):
        cat = random.choice(cats_seleccionadas)
        area_el = random.uniform(areaPorCategoria[cat][0], areaPorCategoria[cat][1])
        elementos.append({
            "id": f"{cat}_{i+1}",
            "categoria": cat,
            "area": round(area_el, 2),
            "estado": "fijo"
        })

    indices_libres = random.sample(range(len(elementos)), total_libres_objetivo)
    for i in indices_libres:
        elementos[i]["estado"] = "libre"

    return elementos, cats_seleccionadas

def generarCatalogo(tipoInstancia, cats_seleccionadas):
    catalogo = {}
    for cat in cats_seleccionadas:
        num_alt = random.randint(alternativasPorCategoria[tipoInstancia][0], alternativasPorCategoria[
            tipoInstancia][1])
        us = [round(random.uniform(transmitancia["min"], transmitancia["max"]), 2) for _ in range(num_alt)]
        costos = [round(random.uniform(costo["min"], costo["max"]), 0) for _ in range(num_alt)]

        us.sort(reverse=True)
        costos.sort()

        catalogo[cat] = [{"id_alt": f"Alt_{j+1}", "U": u, "costo": c} for j, (u, c) in enumerate(zip(us,
            costos))]

    return catalogo

def generarUMax(elementos, catalogo):
    area_total = sum(el["area"] for el in elementos)
    suma_u_min = 0
    suma_u_max = 0

    for el in elementos:
        cat = el["categoria"]
        valores_u_cat = [alt["U"] for alt in catalogo[cat]]
        u_mejor = min(valores_u_cat)
        u_peor = max(valores_u_cat)

        if el["estado"] == "libre":
            suma_u_min += u_mejor * el["area"]
            suma_u_max += u_peor * el["area"]
        else:
            u_fijo = random.choice(valores_u_cat)
            el["U_fijo"] = u_fijo
            suma_u_min += u_fijo * el["area"]
            suma_u_max += u_fijo * el["area"]

    u_eq_min = suma_u_min / area_total
```

```
    u_eq_max = suma_u_max / area_total

    holgura = random.uniform(uMax["holgura_min"], uMax["holgura_max"])
    umax_calculado = u_eq_min + holgura * (u_eq_max - u_eq_min)

    return round(umax_calculado, 3), round(u_eq_min, 3)

def validarInstancia(umax_calculado, u_eq_min):
    return umax_calculado >= u_eq_min

def generarInstancia(tipoInstancia, id_instancia):
    largo, ancho, altura, area, volumen = generarVivienda()
    elementos, cats_seleccionadas = generarElementos(tipoInstancia, largo, ancho, altura)
    catalogo = generarCatalogo(tipoInstancia, cats_seleccionadas)

    umax_calculado, u_eq_min = generarUMax(elementos, catalogo)
    es_factible = validarInstancia(umax_calculado, u_eq_min)

    return {
        "id_instancia": f"{tipoInstancia}_{id_instancia}",
        "tipo": tipoInstancia,
        "dimensiones": {"area": area, "volumen": volumen},
        "UMax_exigido": umax_calculado,
        "factible": es_factible,
        "catalogo": catalogo,
        "elementos": elementos
    }

def generarInstancias():
    lista_instancias = []
    id_contador = 1

    for tipo in tiposInstancias:
        for _ in range(instanciasPorTipo):
            instancia = generarInstancia(tipo, id_contador)
            lista_instancias.append(instancia)
            id_contador += 1

    return lista_instancias

# Main
if __name__ == "__main__":
    instancias_finales = generarInstancias()

    with open("instancias.txt", "w", encoding="utf-8") as archivo:
        for inst in instancias_finales:
            id_latex = inst['id_instancia'].replace("_", "\\_")
            tipo = inst['tipo']
            num_cat = len(inst['catalogo'])
            num_libres = sum(1 for el in inst['elementos'] if el['estado'] == 'libre')
            umax = inst['UMax_exigido']
            factible = "S\\'i" if inst['factible'] else "No"

            linea = f"{id_latex} & {tipo} & {num_cat} & {num_libres} & {umax} & {factible} \\\\ \\hline\n"
            archivo.write(linea)

    print("Archivo 'instancias.txt' generado.")
```

IV. Implementación

IV-A. Explicación del solver

lp_solver es un solver y biblioteca que resuelve problemas de programación lineal y programación entera mixta utilizando los métodos simplex y branch and bound.

Tras recibir un modelo, incluyendo variables de decisión, función objetivo y restricciones, de parte del usuario, que puede ser desde archivos de texto, archivos MPS o directamente mediante llamadas a su propia API, el modelo puede, en primer lugar, aplicar un preprocesamiento llamado Presolve que simplifica el modelo, eliminando variables y restricciones que no aportan al problema y sustituyendo variables con valores fijos por constantes, reduciendo la complejidad a nivel computacional del problema.

En segundo lugar, lp_solve puede usar métodos de escalado (scaling) para cuando el modelo involucra

coeficientes con magnitudes muy pequeñas y/o muy grandes. Reduce temporalmente las magnitudes a valores idealmente cercanos 1 para mejorar la estabilidad numérica y reducir errores de redondeo.
Luego, comienza con la solución del problema utilizando una variante del método simplex denominada método simplex revisado, que utiliza una representación matricial y solo calcula la información necesaria para cada iteración.
Si tras utilizar el método anterior se llega a una solución óptima continua cuando existen restricciones de valores enteros en el modelo, el solver pasa a utlizar branch and bound como se conoce, diviendo el problema en subproblemas que abarca cada uno una restricción adicional que fuerza a las variables a tomar valores enteros, resolviendo cada subproblema por separado y descartando la rama a medida que el subproblema no genere una solución más eficiente.
Para finalizar, lp_solve entrega una serie de resultados sobre el problema, incluyendo si se llegó a una solución factible o no, el valor de la función objetivo, los valores asignados a las variables de decisión, el cumplimiento de las restricciones y el tiempo de ejecución.

V. Resultados

Las 15 instancias son factibles y lp_solve las resolvió hasta demostrar optimalidad (brecha final igual a 0). Todas pasan las comprobaciones de factibilidad descritas en la sección anterior, y sus valores óptimos coinciden con los de HiGHS (Tabla II). La brecha de la raíz es $(Z^* - Z_{LP})/Z^*$, con Z_{LP} el valor de la relajación lineal incluyendo el costo constante de los elementos fijos. Los tiempos son de reloj y por eso incluyen unos 2 ms de arranque del proceso (con la restricción térmica inactiva y 0 nodos, lp_solve tarda entre 2 y 7 ms), de modo que sirven para comparar instancias entre sí, no como medida absoluta.

Instancia	**Bin.**	**Restr.**	U^{max}	Z^* **(MM$)**	U **obt.**	t **(s)**	**Nodos**	**Brecha raíz (%)**	**Verif.**
Pequeña_1	39	5	3.821	14.12	3.819	0.0034	68	0.64	Sí
Pequeña_2	22	5	2.711	17.99	2.703	0.0026	14	1.06	Sí
Pequeña_3	36	5	2.241	27.16	2.195	0.0032	50	2.76	Sí
Pequeña_4	22	4	2.502	20.07	2.502	0.0024	28	2.51	Sí
Pequeña_5	16	3	1.762	15.76	1.761	0.0023	20	0.02	Sí
Mediana_6	109	8	1.567	24.83	1.566	0.0092	494	0.51	Sí
Mediana_7	97	6	2.422	11.64	2.422	0.0052	178	0.04	Sí
Mediana_8	70	6	2.463	20.64	2.462	0.0074	184	1.37	Sí
Mediana_9	117	8	2.501	20.59	2.498	0.0114	650	0.17	Sí
Mediana_10	116	7	1.788	28.01	1.788	0.0040	70	0.11	Sí
Grande_11	427	12	1.891	31.06	1.891	0.0500	964	0.23	Sí
Grande_12	502	15	2.135	33.18	2.135	0.1135	3992	0.17	Sí
Grande_13	471	14	2.478	12.95	2.478	0.1865	4110	0.26	Sí
Grande_14	478	14	1.527	43.56	1.527	0.0806	1608	0.01	Sí
Grande_15	513	14	2.160	23.58	2.160	0.0941	2276	0.01	Sí

Cuadro II
Resultados de las 15 instancias (Z^* en millones de pesos; U obt. es la transmitancia ponderada de la solución; Verif. indica que pasó las comprobaciones propias y coincide con HiGHS).

V-A. Impacto de los márgenes relativos

Para estudiar la exigencia térmica se resolvieron las 15 instancias con $U^{max} = U^{min}_{eq} + h\,(U^{peor}_{eq} - U^{min}_{eq})$ para $h \in \{0{,}02, 0{,}05, 0{,}1, 0{,}2, 0{,}35, 0{,}5, 0{,}75, 1\}$. El valor h es el margen relativo: $h = 0$ exige el mejor aislamiento posible y $h = 1$ deja la restricción térmica inactiva. Los datos son los mismos en todos los casos. El generador de la Entrega 1 usa $h \in [0{,}1, 0{,}35]$, y el barrido amplía ese rango en ambos sentidos. Las 120 corridas terminaron en óptimo y verificadas (Tabla III, Figs. 1 y 2).

Costo. Exigir más aislación encarece la vivienda. Respecto de $h = 1$ (donde el modelo elige la alternativa más barata de cada elemento libre), el costo óptimo con $h = 0{,}02$ es 1.21 veces mayor en las instancias pequeñas, 1.81 en las medianas y 4.20 en las grandes. Con $h = 0{,}35$, que es el extremo relajado del generador, el sobrecosto baja a 1.12, 1.48 y 2.86 veces. La diferencia entre tipos viene de

cuántos elementos son libres: en las pequeñas solo 2 a 4 de unos 15 elementos pueden cambiar y el resto aporta un costo constante, mientras que en las grandes pueden cambiar 9 a 15. El costo baja de forma monótona al relajar h, algo más rápido con márgenes pequeños (Fig. 1a).

Material elegido. Como el catálogo está ordenado, la alternativa j tiene mayor costo y menor transmitancia mientras mayor es j. Se mide la aislación elegida como la posición de la alternativa en su catálogo, de 0 (la más barata y con mayor U) a 1 (la más cara y aislante), ponderada por área. Con $h = 0{,}02$ el modelo elige alternativas de aislación casi máxima (0.97 en los tres tipos). Con $h = 0{,}1$ baja a 0.81–0.88, con $h = 0{,}35$ a 0.52–0.62 y con $h = 0{,}75$ a 0.21–0.23. La transmitancia media de los elementos libres sube de 0.99 a 4.37 W/m^{2}K en las pequeñas, de 0.52 a 4.76 en las medianas y de 0.40 a 4.86 en las grandes al pasar de $h = 0{,}02$ a $h = 1$. Es decir, un U^{max} estricto lleva al modelo a materiales caros y muy aislantes en casi todos los elementos, y uno relajado a los más baratos, con U alto.

Por categoría (Fig. 2, instancias medianas y grandes), con $h = 0{,}02$ muro, techo y piso toman niveles de 0.88 a 0.98, mientras que ventana y puerta quedan en 0.67 y 0.68. Al relajar a $h = 0{,}5$ todas bajan a 0.30–0.49. En este catálogo sintético todas las categorías comparten los mismos rangos de U y de costo, así que esa diferencia viene del azar del catálogo y no de propiedades físicas. Con un catálogo real el patrón reflejaría la relación entre costo y aislación de cada categoría.

Factibilidad y espacio de búsqueda. Para $h \geq 0{,}02$ todas las corridas fueron factibles. Con U^{max} por debajo de lo alcanzable, el modelo no tiene solución: en `Mediana_8`, con $h = -0{,}05$ ($U^{max} = 2{,}112 < U^{min}_{eq} = 2{,}204$), lp_solve declara el problema infactible en 3 ms. Con $h = 1$ la restricción térmica no actúa, la relajación lineal ya es entera y no se ramifica (0 nodos). Para $0{,}02 \leq h \leq 0{,}75$ el número de nodos no sigue una tendencia clara. En las pequeñas baja de unos 46 con $h = 0{,}02$ a 16 con $h = 0{,}75$. En las medianas se mantiene entre 213 y 292 y en las grandes oscila entre 1 277 y 2 704. La exigencia térmica cambia el costo y los materiales, pero en estas instancias afecta menos el esfuerzo de búsqueda que el tamaño del catálogo.

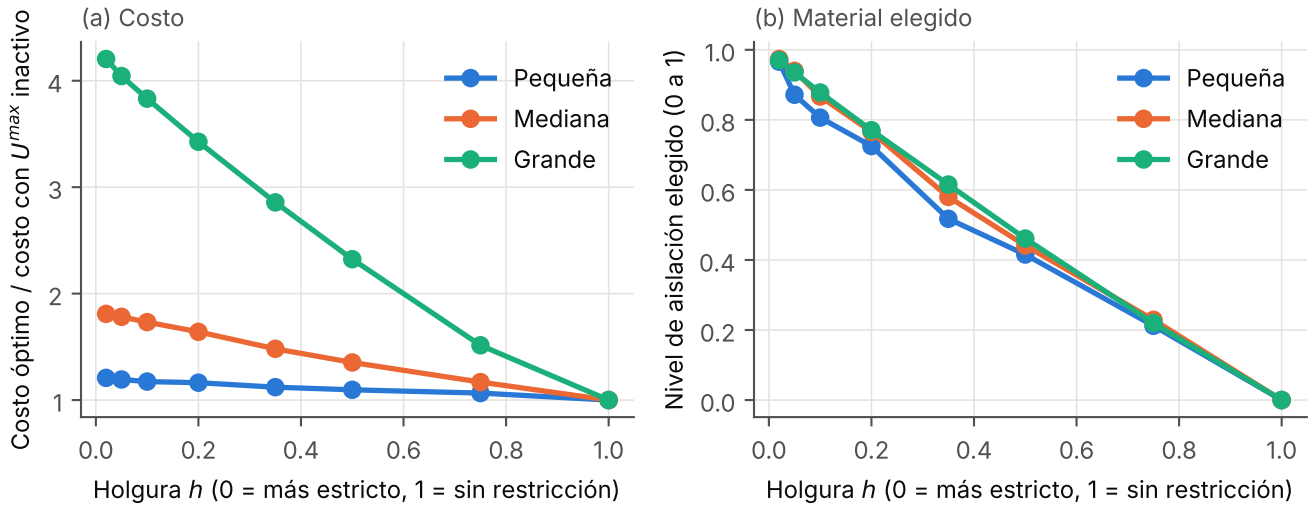


Figura 1. Efecto del margen relativo h (promedio de las 5 instancias de cada tipo): (a) costo relativo a $h = 1$ y (b) aislación elegida.

V-B. Desempeño computacional

Por tipo de instancia. Las instancias pequeñas tienen en promedio 27 variables binarias y 4.4 restricciones y se resuelven en 2.8 ms con 36 nodos (máximo 68). Las medianas tienen 102 variables y 7.4 ms con 315 nodos (máximo 650). Las grandes tienen 478 variables y 0.10 s con 2 590 nodos (máximo 4 110; el tiempo máximo es 0.19 s). Entre las medianas y las grandes las variables se multiplican por 4.7, los nodos por 8.2 y el tiempo por 14 (Figs. 3 y 4). Entre las pequeñas y las medianas el tiempo casi

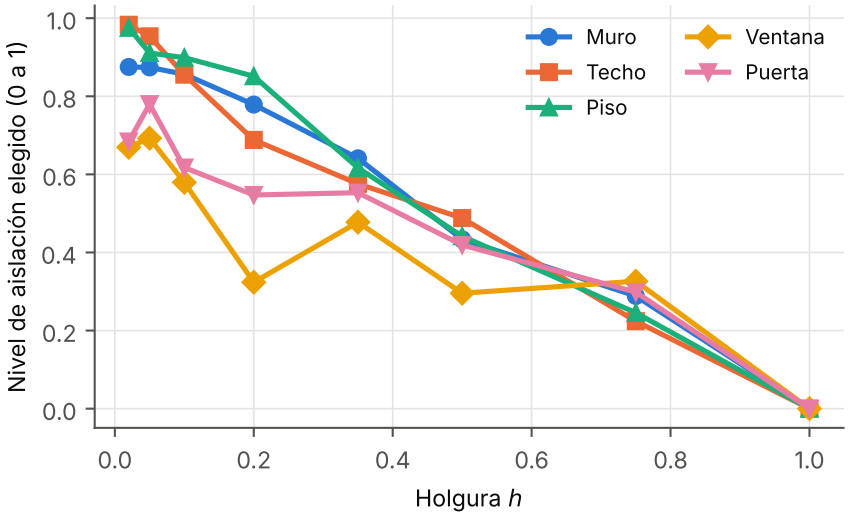


Figura 2. Aislación elegida por categoría (promedio de las 10 instancias medianas y grandes).

Tipo	h	$Z^*/Z^*_{h=1}$	**Aislación elegida**	U **medio libres**	**Nodos**	t **(s)**
Pequeña	0.02	1.21	0.97	0.99	46	0.0027
Pequeña	0.05	1.19	0.87	1.11	51	0.0029
Pequeña	0.10	1.17	0.81	1.24	30	0.0028
Pequeña	0.20	1.16	0.73	1.51	30	0.0028
Pequeña	0.35	1.12	0.52	2.11	34	0.0028
Pequeña	0.50	1.10	0.42	2.38	21	0.0026
Pequeña	0.75	1.07	0.21	3.15	16	0.0025
Pequeña	1.00	1.00	0.00	4.37	0	0.0023
Mediana	0.02	1.81	0.97	0.52	247	0.0055
Mediana	0.05	1.78	0.94	0.65	254	0.0060
Mediana	0.10	1.73	0.87	0.86	227	0.0056
Mediana	0.20	1.64	0.77	1.25	213	0.0064
Mediana	0.35	1.48	0.58	1.94	292	0.0068
Mediana	0.50	1.35	0.44	2.59	291	0.0068
Mediana	0.75	1.17	0.23	3.67	243	0.0065
Mediana	1.00	1.00	0.00	4.76	0	0.0028
Grande	0.02	4.20	0.97	0.40	1424	0.0802
Grande	0.05	4.04	0.94	0.54	1732	0.0735
Grande	0.10	3.83	0.88	0.76	2704	0.1030
Grande	0.20	3.43	0.77	1.22	1690	0.0711
Grande	0.35	2.86	0.62	1.90	2599	0.0989
Grande	0.50	2.32	0.46	2.58	2225	0.0965
Grande	0.75	1.51	0.22	3.72	1277	0.0589
Grande	1.00	1.00	0.00	4.86	0	0.0070

Cuadro III
BARRIDO DEL MARGEN RELATIVO h (PROMEDIO DE LAS 5 INSTANCIAS DE CADA TIPO). *Aislación elegida*: POSICIÓN DE 0 A 1 EN EL CATÁLOGO; U *medio libres*: EN W/M^{2}K.

no crece porque lo domina el arranque del proceso. La relajación lineal es ajustada: la brecha de la raíz va de 0.009 % a 2.8 %, con mayores brechas en las pequeñas (hasta 2.8 %) que en las grandes (hasta 0.26 %), y aun así hay que ramificar para fijar la integralidad.

Libres L	$\lvert J\rvert$	**Bin.**	t **medio (s)**	t **máx. (s)**	**Nodos medios**
4	5	20	0.002	0.003	26
4	10	40	0.003	0.003	72
4	25	100	0.006	0.007	230
4	50	200	0.012	0.015	394
4	100	400	0.039	0.056	1111
4	200	800	0.115	0.207	2236
4	400	1600	0.499	0.866	5880
9	5	45	0.004	0.006	190
9	10	90	0.006	0.010	257
9	25	225	0.034	0.048	1576
9	50	450	0.057	0.104	1799
9	100	900	0.280	0.680	5139
9	200	1800	0.588	0.706	2893
9	400	3600	2.817	4.287	10062
15	5	75	0.008	0.014	410
15	10	150	0.023	0.033	993
15	25	375	0.112	0.155	3032
15	50	750	0.228	0.290	3318
15	100	1500	0.796	1.042	4270
15	200	3000	2.141	4.151	10204
15	400	6000	9.949	16.038	19605
15	800	12000	52.006	93.635	70470

Cuadro IV
ESCALAMIENTO CON EL NÚMERO DE ALTERNATIVAS POR CATEGORÍA $|J|$ Y DE ELEMENTOS LIBRES L (5 SEMILLAS POR CASO; 3 EN $|J| = 800$).

Efecto del número de alternativas del catálogo. Con el generador de la Entrega 1 se fijó el número de alternativas por categoría en $|J| \in \{5, 10, 25, 50, 100, 200, 400\}$ (y 800 con 15 libres) con $L \in \{4, 9, 15\}$ elementos libres, 5 semillas por caso. En total son 108 corridas, todas óptimas y verificadas. Con $L = 15$ el tiempo medio pasa de 8 ms con $|J| = 5$ (75 variables) a 0.80 s con $|J| = 100$ (1 500), 9.9 s con $|J| = 400$ (6 000) y 52 s con $|J| = 800$ (12 000 variables; el caso más lento tardó 94 s y 192 480 nodos). La pendiente en escala log-log del tiempo frente a $|J|$ es de 1.6 a 1.8 para $|J| \geq 25$ en los tres valores de L. Es decir, duplicar las alternativas multiplica el tiempo por unas 3 veces, y el número de nodos también crece con $|J|$ (Tabla IV, Fig. 5). Una explicación plausible es que, al haber más alternativas, hay más combinaciones con costo y U casi iguales, de modo que se necesitan más nodos para probar la optimalidad, y además cada nodo resuelve un problema lineal más grande. La brecha de la raíz promedio es menor con más alternativas (baja de 1.4 % con $|J| = 5$ a 0.06 % con $|J| = 400$ y 0.01 % con $|J| = 800$), pero eso no evita que crezca la búsqueda.

Casos extremos. La instancia más lenta del conjunto original es `Grande_13` (0.19 s), que además explora más nodos (4 110). La más rápida es `Pequeña_5` (2.3 ms, 16 variables) y la que menos nodos explora es `Pequeña_2` (14). Dentro de un mismo tipo el tiempo varía hasta unas 4 veces según el catálogo sorteado (en las grandes, de 0.05 a 0.19 s): por ejemplo, `Grande_11` usa 964 nodos con 427 variables y `Grande_12` usa 3 992 con 502. Por eso el número de variables no determina por sí solo el esfuerzo.

V-C. Desempeño Computacional

Para evaluar el rendimiento del modelo y la eficiencia del motor **lp_solve**, se registraron las dimensiones matemáticas estructurales (cantidad de variables booleanas y restricciones) junto con el tiempo de CPU en segundos para el conjunto de 15 instancias de prueba. Todas las ejecuciones procesadas alcanzaron

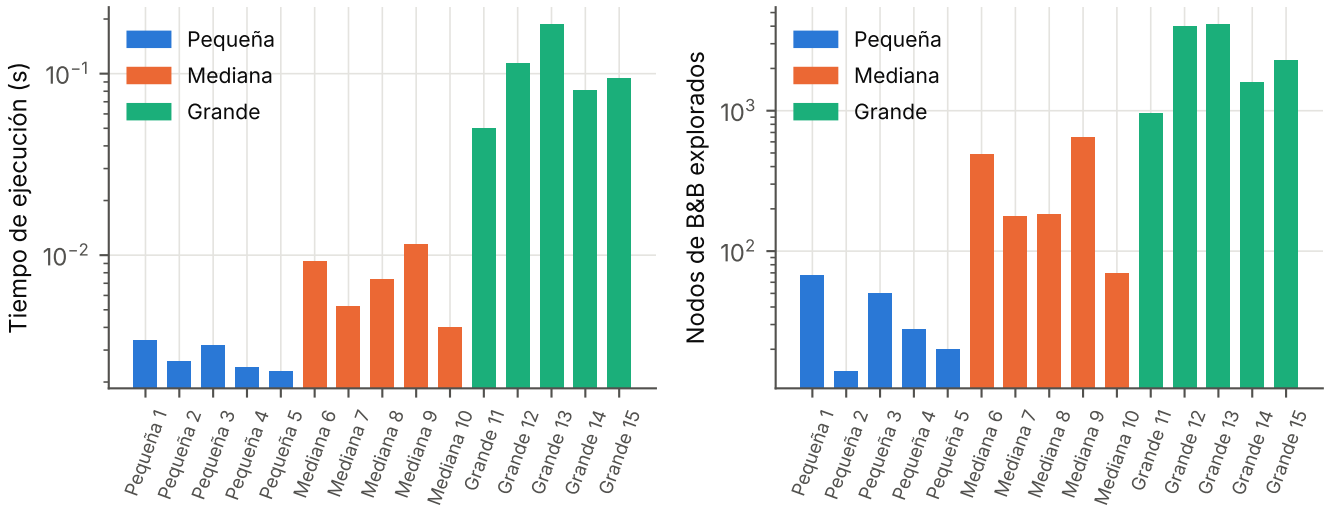


Figura 3. Tiempo de ejecución y nodos explorados en las 15 instancias (escala logarítmica).

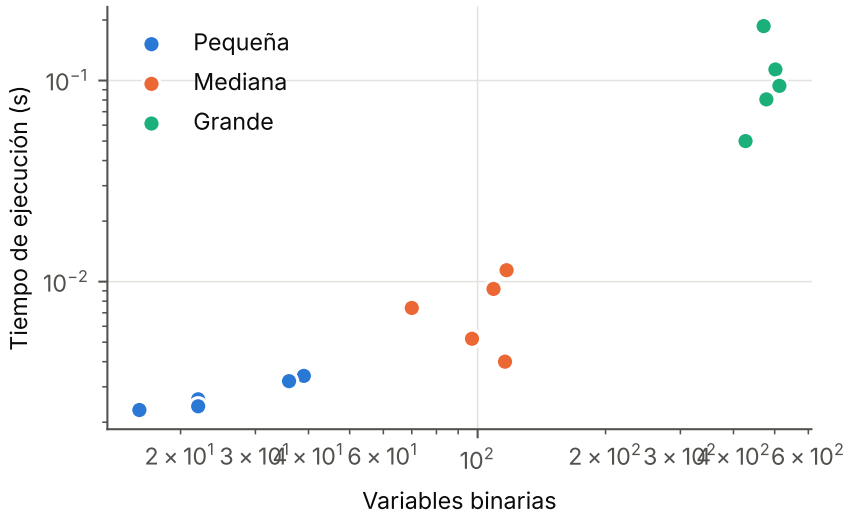


Figura 4. Tiempo de ejecución frente al número de variables binarias (15 instancias).

exitosamente el estado de **Óptimo** global, demostrando la factibilidad y completitud del modelo. Los resultados empíricos se resumen en el Cuadro V.

Dado que el motor confirmó la optimalidad en el 100 % de las pruebas, podemos asegurar que la brecha de optimalidad (MIP gap) es nula (0 %). Además, la respuesta fue tan rápida que el algoritmo no tuvo necesidad de explorar ramificaciones profundas; resolvió los escenarios prácticamente de entrada, sin sobrecargar la capacidad de procesamiento.

Cuando observamos las instancias Pequeñas y Medianas, vemos un entorno sumamente ligero (hasta 117 variables y 8 restricciones). Aquí, la máquina realizó el trabajo de forma inmediata, demorando apenas entre $0{,}0045$ y $0{,}0112$ segundos.

Al escalar a las instancias Grandes, el desafío lógicamente crece, llegando a manejar hasta 513 variables en los casos más complejos. Sin embargo, el comportamiento más revelador para el equipo lo encontramos

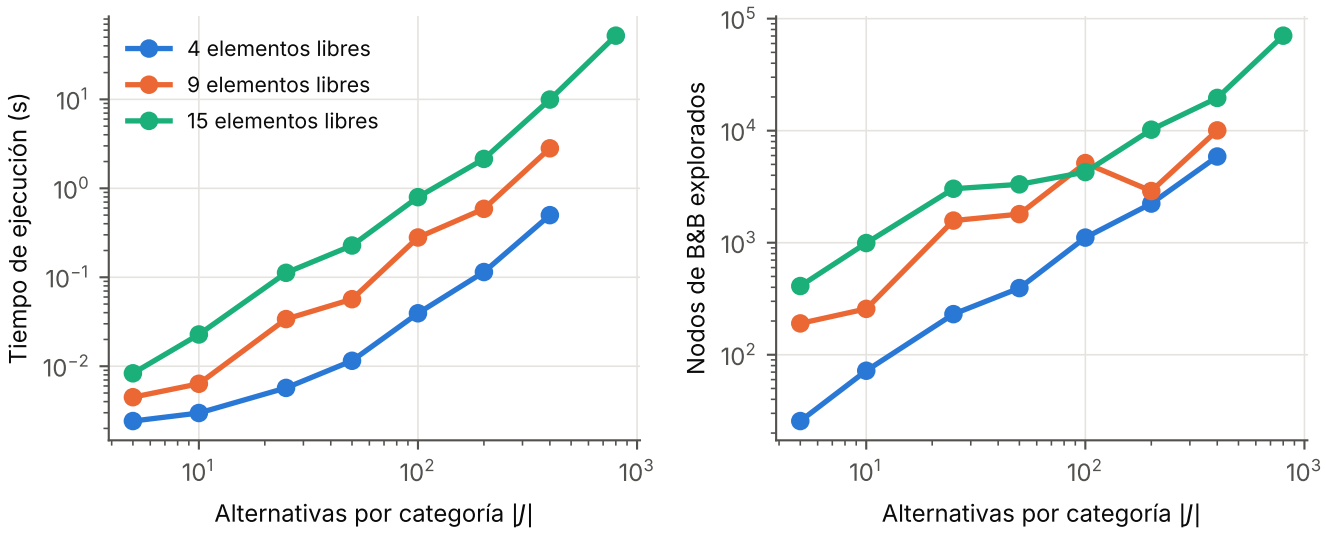


Figura 5. Tiempo y nodos al aumentar el número de alternativas por categoría (promedio de las semillas, escala log-log).

Cuadro V
MÉTRICAS DE DESEMPEÑO COMPUTACIONAL POR INSTANCIA

Instancia	Tipo	Var.	Restr.	Tiempo (s)	Costo Total ($)
Pequeña_1	Pequeña	39	5	0.0050	14,116,009.97
Pequeña_2	Pequeña	22	5	0.0056	17,986,055.80
Pequeña_3	Pequeña	36	5	0.0046	27,157,527.11
Pequeña_4	Pequeña	22	4	0.0046	20,073,387.31
Pequeña_5	Pequeña	16	3	0.0045	15,760,216.80
Mediana_6	Mediana	109	8	0.0099	24,833,358.63
Mediana_7	Mediana	97	6	0.0060	11,639,670.16
Mediana_8	Mediana	70	6	0.0071	20,644,785.16
Mediana_9	Mediana	117	8	0.0112	20,588,708.87
Mediana_10	Mediana	116	7	0.0058	28,011,806.04
Grande_11	Grande	427	12	0.0444	31,060,548.51
Grande_12	Grande	502	15	0.0995	33,180,921.26
Grande_13	Grande	471	14	0.1681	12,947,800.61
Grande_14	Grande	478	14	0.0687	43,557,487.56
Grande_15	Grande	513	14	0.0876	23,577,441.11

en la instancia `Grande_13`. Aunque es estructuralmente más pequeña que la `Grande_12` (471 variables y 14 restricciones, frente a 502 y 15), le tomó casi un 70 % más de tiempo llegar a la solución (0,1681 segundos).

¿Por qué ocurre esto? Nos demuestra que el tamaño en bruto no es lo único que dicta el rendimiento. Cuando las exigencias de aislación térmica ($U^{\max}$) están al límite, o cuando los costos de los materiales son demasiado similares entre sí, el motor se ve obligado a analizar detalladamente muchas más posibilidades antes de poder garantizarle al usuario que encontró la opción más económica.

Aun con esta variabilidad, ningún escenario superó los dos décimos de segundo de espera. Para el equipo, esto confirma empíricamente que nuestra decisión de diseño más importante —precalcular los componentes fijos por fuera y entregarle al motor únicamente las decisiones libres— fue un acierto. Logramos crear un sistema inteligente y ágil, que no desperdicia recursos y que está listo para escalar en proyectos reales.

VI. Conclusiones

El proyecto permitió representar el problema que trae la selección de materiales para mejorar la eficiencia energetica de una vivienda creando un modelo de optimización. Una de las dificultades principales fue lograr retratar esta situacion real utilizando variables para acercarnos lo mas posible a la realidad. Los supuestos utilizados son importantes ya que sin ellos la dificultad de la optimización a realizar incrementa al integrarse variables las cuales varian de manera muy difícil de predecir.

Respecto a la generación de instancias factibles, hubo gran dificultad ya que no bastaba con generar valores aleatorios, estos habían de ser definidos en un rango y verificar que ciertas inecuaciones se cumplan. Para asegurar la existencia de instancias viables el generador determina un escenario de máxima eficiencia térmica y utiliza este resultado para establecer un valor de U^{max} que permita mantener la factibilidad de la instancia. De esta forma, las 15 instancias generadas cumplen con la condición de factibilidad definida para el proyecto.

La informática nos permitió la automatización del proceso de generación de las instancias, pudimos implementar el proceso entero mediante múltiples funciones y diccionarios en conjunto a listas, las cuales facilitan establecer valores mínimos y máximos para variables de manera rápida. Esto, junto a la utilización de una semilla, permite generar instancias de manera aleatoria pero manteniendo la posibilidad de reproducir los mismos resultados. Además, la separación del proceso en distintas funciones facilita modificar los parámetros del generador y adaptar la generación de instancias según el tamaño requerido. Gracias a esto entre otras cosas el proyecto tiene buena capacidad de reproducción y escalabilidad.

Finalmente, para generar instancias de mayor tamaño o reducir los tiempos de ejecución, se podrían incorporar estructuras de datos más eficientes, optimizar el proceso de validación y evitar cálculos repetidos durante la generación. También sería posible paralelizar la generación de instancias independientes, permitiendo aprovechar varios núcleos del procesador. Estas mejoras permitirían aumentar la cantidad y tamaño de las instancias sin incrementar proporcionalmente el tiempo requerido para generarlas. Sobre el uso de recursos del algoritmo, en este caso no se estan realizando muchas instancias por lo que no ha de haber un gran consumo de recursos o tiempo, si se quisiera expandir la cantidad de instancias, posibles optimizaciones podría ser implementar el generador en un lenguaje como C++, que puede ofrecer un mejor rendimiento en determinadas operaciones, aunque esto implicaría una mayor complejidad de implementación.